Group 19

Java O-O Program Visualizer/Animator

Software Design Document

Names:
Prottasha Deb
Angelina Martis
Arya Patel
Aiden Stensrud

Section:
CSE 4101 Section 1

Date: 09/28/2026

TABLE OF CONTENTS

1. INTRODUCTION **3**

1. Purpose 3

2. Scope 3

3. Overview 3

4. Reference Material 3

5. Definitions and Acronyms 3

2. SYSTEM OVERVIEW **4**

3. SYSTEM ARCHITECTURE **4**

1. Architectural Design 4

2. Decomposition Description 5

3. Design Rationale 7

4. DATA DESIGN **7**

1. Data Description 7

2. Data Dictionary 8

5. COMPONENT DESIGN **8**

6. HUMAN INTERFACE DESIGN **9**

1. Overview of User Interface 9

2. Screen Images 10

3. Screen Objects and Actions 10

7. REQUIREMENTS MATRIX **11**

1. INTRODUCTION

1. Purpose

This software design document describes the architecture and system design of the Java Object-Oriented Program Visualizer and Animator.

2. Scope

Students in early Java courses often initially struggle with the concepts of classes and objects. These concepts can be hard to grasp because the inner workings of a program are both "invisible" to the programmer and occur rapidly during execution. The Java O-O Visualizer/Animator aims to visualize the internal memory of a Java program through human-understandable animation, making it easier for users to understand the Java language at a fundamental level.

3. Overview

This document presents the proposed desktop architecture, the data shared among its components, and the operations needed to generate and navigate memory diagrams.

Class names and method signatures below describe a proposed implementation contract. The implementation language, GUI framework, exact Java subset, and resource limits remain to be selected. The design does not imply that these components have already been implemented or tested.

4. Reference Material

[1] Holliday, Luginbuhl, "A Guide to Using Memory Diagrams," Western Carolina University. https://agora.cs.wcu.edu/~holliday/teaching/LectureNotes/150/MemoryDiagramGuide.pdf

[2] Holliday, Luginbuhl, "Using Memory Diagrams When Teaching a Java-Based CS1," Proceedings of the 41st Annual ACM Southeast Conference. https://www.researchgate.net/profile/Mark-Holliday-2/publication/242406239_Using_Memory_Diagrams_When_Teaching_a_Java-Based_CS1/links/0c96052b8ef4e9e9b7000000/Using-Memory-Diagrams-When-Teaching-a-Java-Based-CS1.pdf

[3] Holliday, Luginbuhl, "CS1 Assessment Using Memory Diagrams," Proceedings of 35th SIGCSE Tech Symposium on CS Education. https://dl.acm.org/doi/pdf/10.1145/971300.971373

[4] Appel, Palsberg, "Modern Compiler Implementation in Java," Cambridge University Press. https://eden.dei.uc.pt/~amilcar/pdf/CompilerInJava.pdf

5. Definitions and Acronyms

- O-O: Object-oriented
- SRS: Software Requirements Specification.
- GUI: Graphical user interface.
- AST: Abstract syntax tree, a structured representation of parsed source.

- IR: Intermediate representation: the interpreter instructions generated from a validated AST. These are educational simulation operations, not native machine instructions.
- Execution state: A snapshot of active method frames, variables, represented objects, references, and supported console output.
- Execution trace: The ordered states produced during simulation.
- Frame: The parameters and local variables associated with an active method invocation.
- Object ID: A stable identity within a trace, used to connect references to an object; it is not a physical address.

2. SYSTEM OVERVIEW

The Java O-O Visualizer is a tool for assisting novice coders in visualizing the inner workings of a Java program. Users can input a Java program into the visualizer. The visualizer then parses the Java program, determining how its instructions map to specific system memory instructions. It then uses those system memory instructions to simulate the program's execution. At each step, the system memory is displayed to the user through an easy-to-understand memory diagram. As the program “executes,” the memory diagram will shift to reflect the machine's new state. This allows the user to see how the program’s memory is in its initial state and how it changes as a result of the program's instructions.

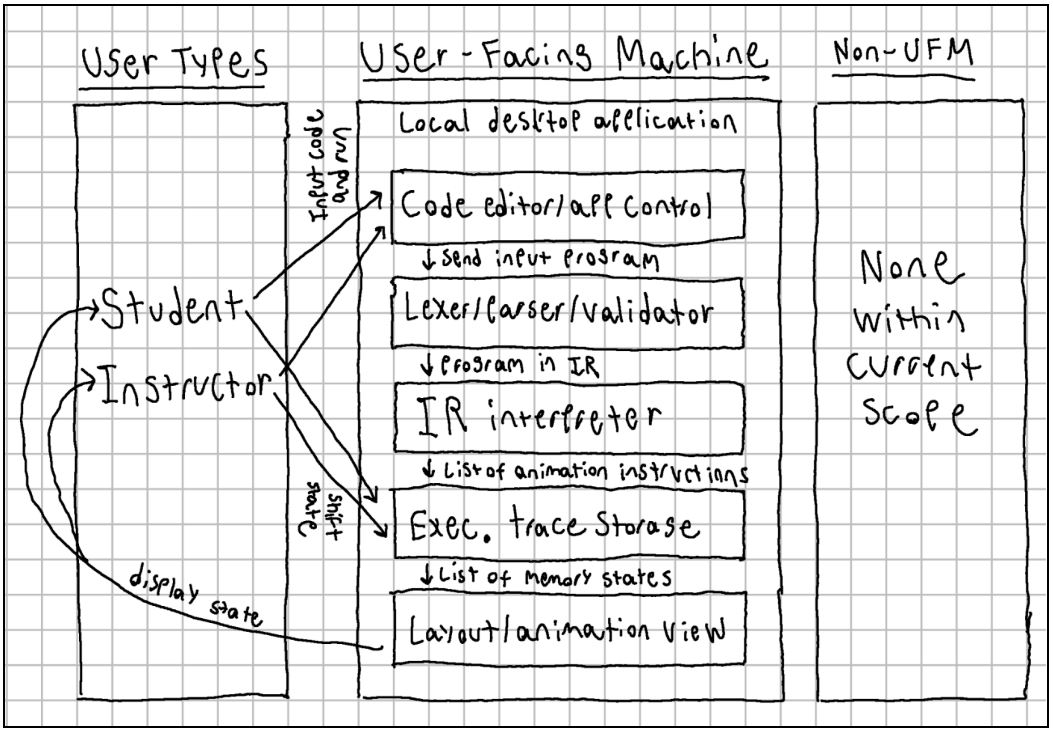
3. SYSTEM ARCHITECTURE

1. Architectural Design

The system uses a layered, locally executed architecture. The presentation layer contains the code editor and diagram view. An application controller coordinates requests. The analysis layer tokenizes and parses source, validates the supported subset, and produces IR instructions. The interpreter generates execution states. The trace store supplies those states to the layout and animation components.

Students and instructors use a desktop computer. Source files are read from local storage; source text, program representations, and trace data are held in application memory. Thus, no database server, remote compiler, or application network protocol is required. While Windows 11 and macOS are the planned test environments, the specific runtime and package dependencies remain open.

The following deployment diagram separates user types, user-facing machines, and non-user-facing machines. All runtime modules belong to the local desktop application. Arrows inside that machine show the main information flow; navigation reads the stored trace rather than rerunning arbitrary Java code.



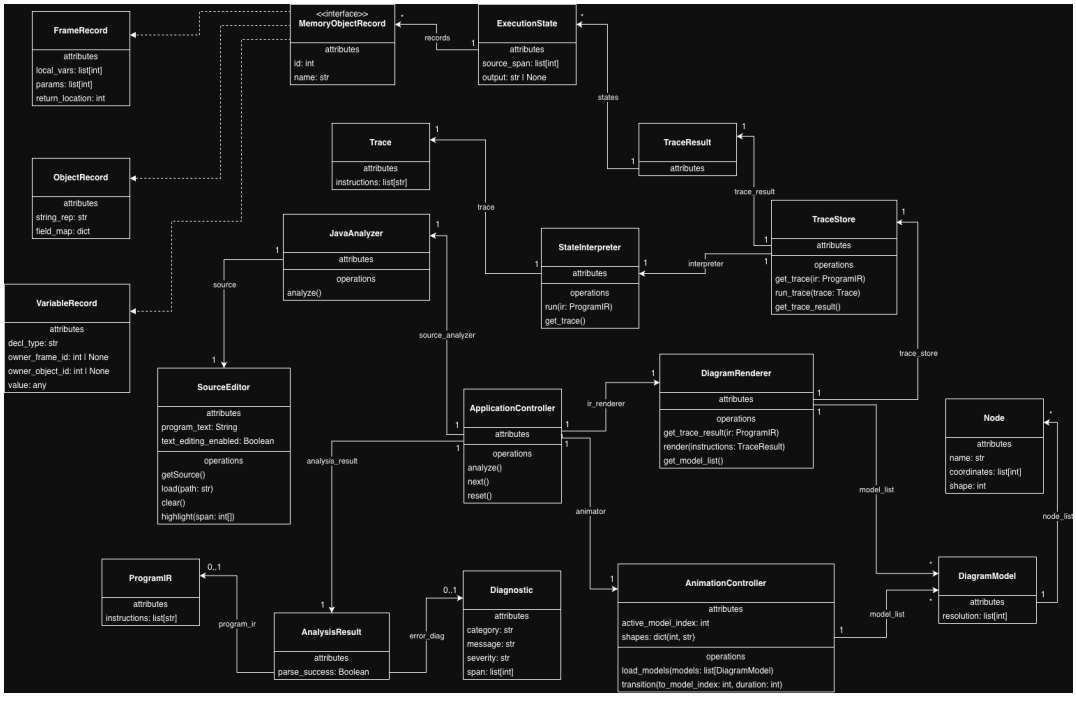
2. Decomposition Description

The following are the major classes within the system decomposition:

- ApplicationController is the central component that coordinates the program's execution.
  - `analyze(source)` coordinates a new analysis
  - `next()` requests the next state
  - `reset()` returns to the initial state.
  - The controller is the only component that commits a change to the selected step. On state change, SourceEditor is told to change highlighted text to the correct span.
- SourceEditor manages visible text, local file loading, line numbers, and highlights.
  - `getSource()` returns a versioned SourceDocument.
  - `load(path)` reads text from a file
  - `clear()` removes text from a file.
  - `highlight(span)` displays the selected source location. File errors are reported to the controller without replacing valid editor content with a partial read.
- JavaAnalyzer contains the lexer, parser, and validation stage.
  - `analyze()` uses the SourceEditor to return an AnalysisResult, which is either ProgramIR instructions or error diagnostics. Validation resolves names and checks the implemented type and feature rules before simulation to ensure program validity. It distinguishes recognized but unsupported constructs from malformed syntax where possible.

- StateInterpreter interprets a set of ProgramIR instructions, converting them into a Trace (list of memory instructions).
    - `run(ir)` converts a ProgramIR into a Trace (list of instructions) which is stored in the object.
    - `get_trace()` returns the stored Trace
- TraceStore “runs” a stored Trace, turning it into a TraceResult which is stored.
    - `load_trace(ir)` loads a trace from its StateInterpreter given a ProgramIR.
    - `run_trace()` turns the current trace into a TraceResult.
    - `get_trace_result()` returns the stored TraceResult.
- DiagramRenderer renders memory diagrams given a TraceResult.
    - `load_trace_result(ir)` loads a TraceResult from the object’s TraceStore given a ProgramIR.
    - `render(instructions)` converts a loaded TraceResult into a set of memory diagrams stored as DiagramModel objects, which is stored in the object.
    - `get_instructions()` returns the object’s loaded list of instructions.
- AnimationController computes and executes presentation changes between memory diagram models.
    - `load_models(list[DiagramModel])` loads a list of DiagramModels into the object for animation and sets the active model to the first in the list.
    - `transition(index: int, duration: int)` switches the displayed model to a different one at the given index with an animation, with the specified duration.

The classes interact with each other in a manner consistent with the below class diagram.



3. Design Rationale

Separating analysis, simulation, and presentation supports independent testing and development. Parser and interpreter cases can compare structured results without opening the GUI. Rendering tests can use prepared states, and integration cases can verify that the displayed state and highlighted source span agree.

Shared references use object IDs so that copying a reference does not copy an object. The trade-off of this approach, however, is that memory usage is proportional to the retained trace data; source, step, and object limits must therefore be measured and agreed upon. Structural sharing or state deltas may be introduced later while retaining the same read-only trace interface.

Long analysis should run outside the UI event loop with cancellation and a source-version check before results are installed. A late result for edited source is discarded. This keeps controls responsive and prevents the diagram from silently representing an older program. The exact concurrency mechanism depends on the selected framework.

Local simulation avoids dependence on a cloud compiler and provides direct access to the educational state. A complete Java runtime is not assumed. Only the agreed language subset and explicitly modeled operations are accepted, which limits library support but makes the displayed semantics testable. Program analysis separation also allows for easier expansion of the subset grammar during further development.

4. DATA DESIGN

1. Data Description

The source program is converted into tokens that carry their source locations. These tokens then become an AST which preserves statement and expression structure. Validation resolves supported declarations and expressions. Lowering produces typed IR operations with operands and SourceSpan metadata. Candidate operations include declaring a variable, assigning a value, allocating an object, reading or writing a field, entering a method, returning, and emitting supported text output.

The interpreter maintains a stack of FrameRecord values and an object map keyed by ObjectId. A variable holds either a primitive value or a ReferenceValue containing an object ID or null. Object fields use the same value representation. Two variables may therefore share the same object ID while referring to the same stored object.

After each defined educational step, the interpreter captures an immutable ExecutionState. Earlier states must not share mutable field or variable containers with the current state. A TraceResult retains the initial state, the ordered states, the source version, and the completion status. Console text is stored by state.

The renderer derives DiagramModel nodes, labels, bounding boxes, and arrow endpoints from the selected state. Layout coordinates belong to presentation data, not the interpreter's memory model. A deterministic layout policy reuses prior positions where possible, expands nodes for field labels, separates active frames from objects, and routes references after placement.

No persistent database is proposed. Local source files are input; traces and layout caches remain in memory for the current analysis. Saving sessions or exporting traces is outside the established core scope.

2. Data Dictionary

- AnalysisResult: An analysis result containing either ProgramIR instructions or a list of Diagnostic records. An invalid input will never produce a successful result.
- Diagnostic: A record containing category, message, severity, and SourceSpan. Categories distinguish syntax, unsupported-feature, semantic, runtime, and resource-limit problems.
- DiagramModel: A model of a memory diagram. It contains a list of Nodes.
- ExecutionState: immutable record containing stateIndex, frames, objects, sourceSpan, explanation, and output. The initial state has no completed source operation.
- FrameRecord: record containing frameId, methodName, returnLocation, parameters, and locals. Stack order identifies the current invocation; unique frame IDs distinguish repeated or recursive calls if supported.
- MemoryObjectRecord: Interface class for ObjectRecord, VariableRecord, and FrameRecord.
- Node: An object in a memory diagram. It has a title (name), coordinates for its placement, and a shape which corresponds to the type of object.
- ObjectRecord: record containing objectId, className, and field map. A string representation additionally stores its text. Object IDs are unique within a trace and are never treated as host addresses.
- ProgramIR: An AST representing an input source program.
- ReferenceValue: tagged value containing ObjectId or null. Null has no target node. Copying this value preserves target identity.
- SourceDocument: record containing text, optional filePath, and monotonically increasing version. SourceSpan: record containing starting and ending line and column positions.
- Trace: a record containing a list of memory diagram instructions.
- TraceResult: a record containing an ordered list of ExecutionStates, the execution status, and an optional Diagnostic. Status is complete, failed, limited, or canceled. A failed or limited trace cannot be labeled complete.
- VariableRecord: record containing variableId, name, declaredType, ownerFrameId or ownerObjectId, and value. Value is a tagged primitive or ReferenceValue; primitive types retain their supported Java semantics.

5. COMPONENT DESIGN

The following procedural descriptions define the proposed interfaces. They describe design behavior rather than completed implementation. Operations use the supported subset of Java.

- ApplicationController.analyze(source): Call JavaAnalyzer.analyze(source). If diagnostics are returned, there was an error in the input code; display it and do not install a success trace. Otherwise, call StateInterpreter.run(program, limits, cancellation). Show the initial state and the returned completion status.

- ApplicationController.next(): Move to the next memory diagram state in the TraceResult.
- ApplicationController.reset(): Reset animation playback and remove current TraceResult.
- ApplicationController.beginEdit(): cancel playback and analysis, invalidate the active trace, and enable text editing.
- SourceEditor.getSource(): return current program text.
- SourceEditor.load(path): read the complete text into a temporary buffer; on success replace editor text and increment version; on failure preserve existing text and report the error.
- SourceEditor.clear(): replace text with an empty string and increment version.
- SourceEditor.highlight(span): clear the old highlight, then mark the requested valid source range or no range for the initial state.
- JavaAnalyzer.analyze(source): perform traditional lexical, syntax, and semantic analysis on the source program to convert it into an AST.
- StateInterpreter.run(ProgramIR): create the initial frame and object store; save state zero; while an operation remains, check cancellation and configured resource bounds, evaluate one supported educational step, update simulated values and frames, and capture an immutable snapshot. On normal return, mark complete. On an error or exhausted limit, return the appropriate status and diagnostic without presenting the result as normal completion.
- TraceStore.load(trace): replace the stored trace with the given trace.
- TraceStore.get(index): get a memory state at a given index
- TraceStore.clear(): discard the TraceResult.
- DiagramRenderer.layout(state, previousLayout): generate a memory diagram DiagramModel given a memory state.
- DiagramRenderer.render(TraceResult): draw a list of memory diagrams given a TraceResult.
- AnimationController.transition(toIndex, duration): executes a transition from the stored model to the toModel model within the given duration.

6. HUMAN INTERFACE DESIGN

1. Overview of User Interface

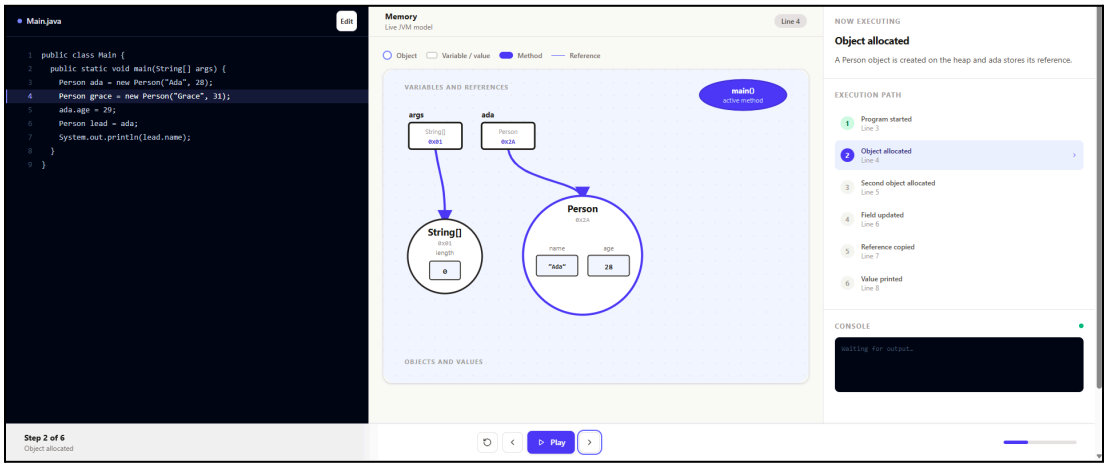
Users enter or load source, request analysis, and inspect the initial diagram. Navigation selects stored execution states and updates values, references, objects, active frames, and explanations together.

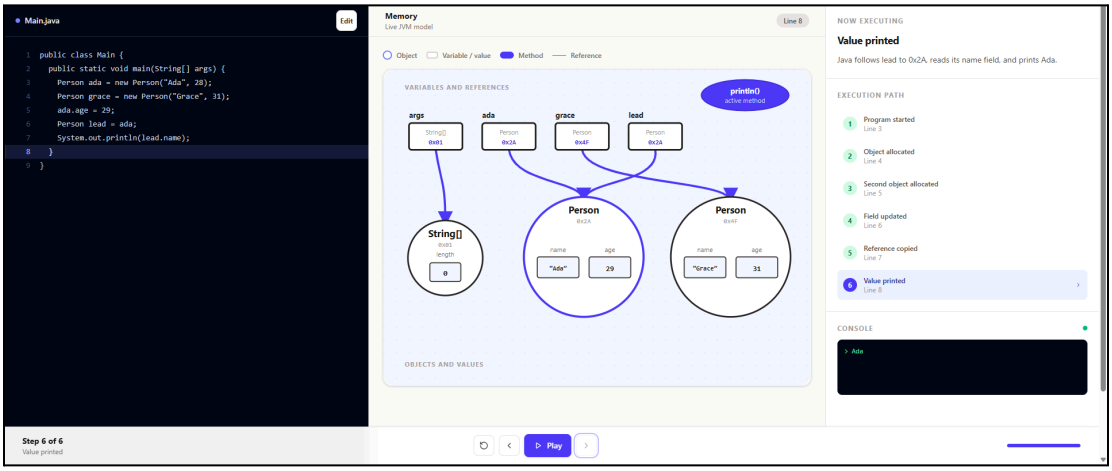
The proposed convention shows the state after the highlighted operation. State zero has no completed operation; one source line may yield several steps. The diagram, highlight, explanation, step indicator, and supported console output share a single selected index.

The supplied mockups retain illustrative line numbers and simplified string fields. Final source spans and string references follow the execution model. “Live JVM model” means educational simulation; object IDs are not physical addresses.

Diagnostics identify source locations when available. Editing invalidates the trace and stops playback. Boundary controls are disabled. Play, console output, and speed adjustment require scope confirmation.

2. Screen Images





3. Screen Objects and Actions

- Code Window - Where the user inputs code to create the diagram
- Restart/Reset - this button will reset the diagram as well as the program
- Back - this button allows the user to go back one line of code at a time and reverse the diagram by one frame
- Forward - this button allows the user to go forward one line of code at a time and update changes in the diagram after executing that line of code
- Play - this button, if used, will iterate through the entire code, changing the diagram automatically as the code is being executed
- Edit - this button allows a user to edit the program they've input, thus updating the diagram as well
- "Now Executing" - this window shows a breakdown of what each line of code is executing
- "Console" - this window will show any output produced from the program, if there is any

7. REQUIREMENTS MATRIX

Requirement	Components	Data
Source input	SourceEditor; ApplicationController	SourceDocument
Analysis	JavaAnalyzer	AnalysisResult; ProgramIR; Diagnostic
State trace	StateInterpreter; TraceStore	ExecutionState; FrameRecord; ObjectRecord
Navigation	ApplicationController; TraceStore	TraceResult; selected index
Code and state	ApplicationController; SourceEditor; AnimationController	SourceSpan; ExecutionState
Memory display	DiagramRenderer	DiagramModel; VariableRecord; ReferenceValue
Readability	DiagramRenderer	Layout bounds; nodes; edges
Animation accuracy	AnimationController; ApplicationController	Immutable states; target DiagramModel

NOTE: Optional playback, transition-speed control, and modeled console output are not separate core functional requirements in the current SRS. They require scope confirmation.